Name	AYUSHI JAPSARE
UID no.	2023301006
EXP	3

CSS	
AIM:	To implement RSA public-key cryptosystem for: i) Key generation (public and private keys) ii) Encryption and Decryption of a message
PROBLEM STATEMENT:	RSA (Rivest–Shamir–Adleman) is an asymmetric cryptographic algorithm based on the computational hardness of integer factorization. The algorithm uses two keys: a public key for encryption/verification and a private key for decryption/signing. The security relies on the difficulty of factoring a large composite number $n = p \times q$. This experiment covers key generation, encryption and decryption. Key Generation – 1. Choose two large distinct primes p and q; 2. Compute $n = p \times q$ and Euler's totient $\varphi(n) = (p-1)(q-1)$. 3. Select an integer e such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$. 4. Compute the modular multiplicative inverse d of e modulo $\varphi(n)$, i.e., $d \equiv e^{-1} \pmod{\varphi(n)}$. 5. The public key is (e, n), and the private key is (d, n). ● Encryption – Given plaintext m ($0 \leq m < n$) and public key (e, n), compute ciphertext $c \equiv m^e \bmod n$. ● Decryption – Given ciphertext c and private key (d, n), compute $m \equiv c^d \bmod n$.
THEORY	The RSA algorithm is a widely used public-key cryptosystem that enables secure data transmission. It is based on the mathematical properties of large prime numbers and the difficulty of factoring large integers. 1. Key Generation (Public and Private Keys) The first step in RSA is to generate the keys: Step-by-step: 1. Choose two distinct large prime numbers, p and q. ○ These should be kept secret.

2. Compute `n = p × q`
    - `n` is the modulus for both the public and private keys.
    - It determines the key length (bit size).

3. Compute Euler's totient function, `φ(n) = (p - 1) × (q - 1)`

4. Choose an integer `e` such that:
    - `1 < e < φ(n)`
    - `gcd(e, φ(n)) = 1` (i.e., `e` and `φ(n)` are co-prime)
    - Common choice: `e = 65537` (fast and secure)

5. Compute the modular inverse `d` of `e` modulo `φ(n)`:
    - `d ≡ e⁻¹ mod φ(n)`
    - `d` is the private exponent

The Keys:

- Public Key: `(e, n)`
- Private Key: `(d, n)`

The public key is shared openly, while the private key is kept secret.

📌 2. Encryption

To encrypt a message **`M`**, convert the plaintext into an integer `m` such that:

```
0 ≤ m < n
```

Then compute the ciphertext **`c`** using the public key `(e, n)`:

```
c = m^e mod n
```

📌 3. Decryption

To decrypt the ciphertext **`c`**, use the private key `(d, n)`:

```
m = c^d mod n
```

Then convert the integer `m` back to the original message `M`.

CODE:

```
   import random
from math import gcd

# ------------------ Prime Utilities ------------------

def is_prime(n, k=5):
   """Miller-Rabin Primality Test"""
   if n <= 3:
      return n == 2 or n == 3
   if n % 2 == 0:
      return False

   # Write n-1 as 2^r * d
   r, d = 0, n - 1
   while d % 2 == 0:
      r += 1
      d //= 2

   for _ in range(k):
      a = random.randrange(2, n - 2)
      x = pow(a, d, n)
      if x in (1, n - 1):
         continue
      for _ in range(r - 1):
         x = pow(x, 2, n)
         if x == n - 1:
            break
      else:
         return False
   return True

def generate_prime(bit_length):
   while True:
      num = random.getrandbits(bit_length)
      num |= (1 << bit_length - 1) | 1  # Ensure it's odd and bit length
      if is_prime(num):
         return num

# ------------------ RSA Key Generation ------------------

def mod_inverse(e, phi):
```

```
    """Extended Euclidean Algorithm for modular inverse"""
    old_r, r = phi, e
    old_s, s = 1, 0
    old_t, t = 0, 1
    while r != 0:
        quotient = old_r // r
        old_r, r = r, old_r - quotient * r
        old_t, t = t, old_t - quotient * t
    if old_t < 0:
        old_t += phi
    return old_t

def generate_keys(bit_length):
    p = generate_prime(bit_length)
    q = generate_prime(bit_length)
    while p == q:
        q = generate_prime(bit_length)

    n = p * q
    phi = (p - 1) * (q - 1)

    # Choose e
    e = 65537
    if gcd(e, phi) != 1:
        e = 3
        while gcd(e, phi) != 1:
            e += 2

    # Compute d
    d = mod_inverse(e, phi)

    print(f"\n🔐 RSA Keys Generated Successfully:")
    print(f"Public Key (e, n):\n  e = {e}\n  n = {n}")
    print(f"Private Key (d, n):\n  d = {d}\n  n = {n}")
    return e, d, n

# ------------------ Encryption and Decryption ------------------

def encrypt_message(message, e, n):
    m = int.from_bytes(message.encode('utf-8'), byteorder='big')
    if m >= n:
        raise ValueError("Message too long for the current key size.")
    c = pow(m, e, n)
    print(f"\n🔒 Encrypted Ciphertext:\n  {c}")
```

```
    return c

def decrypt_message(ciphertext, d, n):
    m = pow(ciphertext, d, n)
    message_bytes = m.to_bytes((m.bit_length() + 7) // 8, byteorder='big')
    plaintext = message_bytes.decode('utf-8')
    print(f"\n🔓 Decrypted Message:\n  {plaintext}")
    return plaintext

# ----------------- Menu System -----------------

def menu():
    e = d = n = None
    while True:
        print("\n====== RSA Cryptosystem Menu ======")
        print("1) Generate RSA Keys")
        print("2) Encrypt Message")
        print("3) Decrypt Message")
        print("4) Quit")
        choice = input("Select an option (1-4): ")

        if choice == '1':
            bit_length = int(input("Enter bit length for prime numbers (e.g. 512): "))
            e, d, n = generate_keys(bit_length)

        elif choice == '2':
            if not e or not n:
                print("Please generate keys first.")
                continue
            plaintext = input("Enter message to encrypt: ")
            try:
                ciphertext = encrypt_message(plaintext, e, n)
            except Exception as err:
                print(f"Error: {err}")

        elif choice == '3':
            if not d or not n:
                print(" Please generate keys first.")
                continue
            try:
                ciphertext = int(input("Enter ciphertext (number): "))
                decrypt_message(ciphertext, d, n)
            except Exception as err:
                print(f"Error: {err}")
```

```
        elif choice == '4':
            print(" Exiting RSA Program.")
            break
        else:
            print(" Invalid choice. Please enter 1–4.")

# ------------------ Run the Program ------------------

if __name__ == '__main__':
    menu()
```

OUTPUT:

Output Clear

```
====== RSA Cryptosystem Menu ======
1) Generate RSA Keys
2) Encrypt Message
3) Decrypt Message
4) Quit
```

```
Select an option (1-4): 1
Enter bit length for prime numbers (e.g. 512): 512

🔐 RSA Keys Generated Successfully:
Public Key (e, n):
  e = 65537
  n = 804320333712358516744416810783184081844887789529523584437548803818195944426876227200258457843713301150152234096448220832341399354486800141897825463423230210715742117046619986410922656290241192889588922033454317899339881773878765238705364495360949200583758737007899310739851930044545181088538474141443089738190
Private Key (d, n):
  d = 389795354060122050907417524868161643399537377103730695108427110762923560598471334545484365771612648699665610374861405646520212779008121508564884485768118908639488316961246356781349977514926054894680017048619374337028849036607372697567777249058725271033994922447806594182090847715631289593202225601237612089538043203337123585167444168107831840818448877895295235844375488038181959444268762272002584578437133011501522340964482208323413993544868001418978254634232302107157421170466199864109226562902411928895889220334543178993398817738787652387053644953609492005837587370078993107398519300445451810885384741414430897381900000
  n = 804320333712358516744416810783184081844887789529523584437548803818195944426876227200258457843713301150152234096448220832341399354486800141897825463423230210715742117046619986410922656290241192889588922033454317899339881773878765238705364495360949200583758737007899310739851930044545181088538474141443089738190
```

```
====== RSA Cryptosystem Menu ======
1) Generate RSA Keys
2) Encrypt Message
3) Decrypt Message
4) Quit
Select an option (1-4): 2
Enter message to encrypt: hello world

🔒 Encrypted Ciphertext:
  42503310016881594256016002423239160519979970791199546089597951854233917677729940520116597450370401959371997863763786156618557363567224327240829746368015003006744710849037897044311294409632884440746215673480043706085959639275237599142132039999755240528375795670466196310320272634800996115311883124075819497670
```

```
====== RSA Cryptosystem Menu ======
1) Generate RSA Keys
2) Encrypt Message
3) Decrypt Message
4) Quit
Select an option (1-4): 3
Enter ciphertext (number):
    42503310016881594256016002423239160519979970791199546089597951854233917677729940520116597450370401959371997863763786156618557363567224327240829746368015003006744710849037897044311294409632884440746215673480043706085959639275237599142132039999755240528375795670466196310320272634800996115311883124075819497670

🔒 Decrypted Message:
  hello world
```

Conclusion:

This experiment successfully implemented the RSA public-key cryptosystem, demonstrating key generation, encryption, and decryption processes. By generating large prime numbers, calculating Euler's totient function, and using modular arithmetic, we derived public and private keys for secure communication. The public key was used for encryption, converting plaintext into ciphertext, and the private key was employed to decrypt the ciphertext back to the original message. The experiment highlighted the importance of large prime numbers and modular exponentiation for RSA's security, showcasing Python's ability to handle arbitrary-precision integers.

.